MAPÚA UNIVERSITY
SCHOOL OF INFORMATION TECHNOLOGY
658 Muralla St., Intramuros, Manila, 1002, Philippines
Tel. No. : +63(2) 247-5000 local 5804
Telefax : +63(2) 891-0800
Web : www.mapua.edu.ph

Predicting IoT Network Attack Types Using Machine Learning with Interactive Model Deployment

Submitted by:

Rob Eugene Dequiñon

Jan Christian Diesta

Dustin Joaquin B. Granada

Sean Jared R. Santos

Yuan Benedict Suarez

August 1, 2025

658 Muralla St., Intramuros, Manila, 1002, Philippines
Tel. No. : +63(2) 247-5000 local 5804
Telefax : +63(2) 891-0800
Web : www.mapua.edu.ph

I. Business Understanding 3
II. Data Understanding 4
Collect Initial Data 4
Data Description 4
Initial Data exploration 6
Identifying Target & Column Types 7
Class Imbalance & Attack Type Distribution 8
Correlation Between Features: Service vs Attack 8
Understanding Feature Importance 9
III. Data Preparation 10
Data Loading 10
Missing Value Handling 10
Duplicate Check and Cleanup 11
Column Name Standardization 11
Final Dataset Status 11
IV. Modeling 11
V. Evaluation 13
Classification Metrics 13
Confusion Matrix Analysis 13
Model Interpretation and Practical Implications 14
VI. Deployment 15
Model Export and Serialization 15
Interactive Prediction Interface with Gradio 15
Colab Hosting and Shareable Access 16
Deployment Outcome 16
References: 17

I. Business Understanding

In this paper, our task is to determine the early determinants for an upcoming attack. With the growing adoption of Internet of Things (IoT) devices and smart systems, cyber threats have become increasingly sophisticated and pervasive. These smart environments generate vast amounts of network traffic and behavioral data, making them both valuable and vulnerable. To address these rising concerns, we aim to proactively identify the early warning signs of malicious activity by analyzing network traffic and usage behavior in real-time. The ultimate goal is to predict potential attacks before they cause harm and to prevent them by safeguarding the affected devices or systems.

This study is guided by several key business questions. First, we explore whether malicious behavior can be detected in real-time from smart devices. Early detection is crucial for minimizing damage and maintaining system integrity. By monitoring patterns in device activity and network behavior, we aim to identify suspicious anomalies as they happen.

Next, we investigate which network features are most predictive of cyber-attacks. Certain indicators—such as packet size, communication frequency, protocol usage, or traffic direction—may be strongly correlated with the onset of an attack. Understanding these relationships can help fine-tune detection systems and improve overall cybersecurity posture.

We also seek to determine which devices are most frequently targeted by attacks. Identifying vulnerable endpoints or high-risk devices can help administrators apply preventive measures more strategically, such as deploying security patches, limiting access, or isolating compromised systems.

Finally, we aim to predict the type of attack before it inflicts damage. By classifying attacks (e.g., DDoS, spoofing, or data exfiltration) as early as possible, we enable more timely and appropriate countermeasures that preserve network functionality and user safety.

To achieve these goals, we focus on the **data mining task of classification**. Specifically, we aim to train a model that can:

- Classify network behavior as benign or malicious
- Predict the type of cyberattack based on early indicators
- Learn patterns associated with targeted devices

Through classification, we hope to develop a practical and intelligent solution for proactive cybersecurity in smart environments.

II. Data Understanding

Collect Initial Data

For the data collection phase, we adopted the standard method of acquiring datasets through direct download from a trusted source. Specifically, we obtained the RT-IoT2022 dataset from the UCI Machine Learning Repository, which we then loaded into a Python notebook environment for preprocessing and analysis.

Data Description

The RT-IoT2022 dataset is a proprietary dataset derived from a real-time IoT infrastructure. It offers a comprehensive resource integrating a wide variety of IoT devices and simulated network attack methodologies. The dataset captures both normal and malicious network behaviors, making it well-suited for research into Intrusion Detection Systems (IDS) and proactive cybersecurity.

Key characteristics of the dataset include:

- Devices Represented: ThingSpeak-LED, Wipro-Bulb, MQTT-Temp
- Simulated Attacks: Brute-Force SSH, DDoS (via Hping and Slowloris), and Nmap scan patterns
- Traffic Monitoring Tools: Zeek (formerly Bro) and Flowmeter, capturing bidirectional network flows

The dataset comprises **85 columns** and **123,117 rows**, with the primary target variable being attack_type, which labels each instance according to the type of activity.

For better interpretability and analysis only for this table, we grouped the features into sets representing similar characteristics. For example, features like (fwd_header_size_tot, fwd_header_size_max ,fwd_header_size_min) are grouped into one feature representing total, minimum, and maximum values. This organized structure enhances browsing and feature selection for model training. Below is the grouped feature table and its corresponding descriptions.

Name	Description
`id.orig_p / id.resp_p`	Origin and response port numbers (Integer)
`proto`	Network protocol used (Categorical)
`service`	Type of service (Continuous)

658 Muralla St., Intramuros, Manila, 1002, Philippines
Tel. No. : +63(2) 247-5000 local 5804
Telefax : +63(2) 891-0800
Web : www.mapua.edu.ph

`flow_duration`	Duration of the network flow (Continuous)
`fwd_pkts_tot` / `bwd_pkts_tot`	Total number of forward and backward packets (Integer)
`fwd_data_pkts_tot` / `bwd_data_pkts_tot`	Total forward/backward data packets (Integer)
`fwd_pkts_per_sec`, `bwd_pkts_per_sec`, `flow_pkts_per_sec`	Packet rate per second (Continuous)
`down_up_ratio`	Ratio of download to upload packets (Continuous)
`fwd_header_size`	Forward header size (tot, min, max) — Integer
`bwd_header_size`	Backward header size (tot, min, max) — Integer
`flow_*_flag_count`	Count of flow flags: FIN, SYN, RST, ACK, CWR, ECE (Integer)
`fwd_*_flag_count` / `bwd_*_flag_count`	PSH, URG flag counts in forward/backward flows (Integer)
`fwd_pkts_payload`	Forward packet payload (min, max, tot, avg, std) — Continuous
`bwd_pkts_payload`	Backward packet payload (min, max, tot, avg, std) — Continuous
`flow_pkts_payload`	Total flow packet payload (min, max, tot, avg, std) — Continuous
`fwd_iat`	Inter-arrival time for forward packets (min, max, tot, avg, std) — Continuous
`bwd_iat`	Inter-arrival time for backward packets (min, max, tot, avg, std) — Continuous
`flow_iat`	Inter-arrival time for the whole flow (min, max, tot, avg, std) — Continuous
`payload_bytes_per_second`	Payload throughput (bytes/sec) (Continuous)
`fwd_subflow_pkts`, `bwd_subflow_pkts`	Packets in subflow (Continuous)

`fwd_subflow_bytes`, `bwd_subflow_bytes`	Bytes in subflow (Continuous)
`fwd_bulk_bytes`, `bwd_bulk_bytes`	Bulk transfer bytes (Continuous)
`fwd_bulk_packets`, `bwd_bulk_packets`	Bulk transfer packets (Continuous)
`fwd_bulk_rate`, `bwd_bulk_rate`	Bulk transfer rate (Continuous)
`active`	Active time metrics (min, max, tot, avg, std) — Continuous
`idle`	Idle time metrics (min, max, tot, avg, std) — Continuous
`fwd_init_window_size`, `bwd_init_window_size`	Initial TCP window sizes (Integer)
`fwd_last_window_size`	Last forward window size (Integer)
`Attack_type`	Target variable — Type of attack (Categorical)
`id`	Unique identifier (Integer)

Initial Data exploration

	Unnamed: 0	id.orig_p	id.resp_p	proto	service	flow_duration	fwd_pkts_tot	bwd_pkts_tot	fwd_data_pkts_tot	bwd_data_pkts_tot	...	active.std	idle.min	idle.max	idle.tot	idle.avg
0	0	38667	1883	tcp	mqtt	32.011598	9	5	3	3	...	0.0	2.972918e+07	2.972918e+07	2.972918e+07	2.972918e+07
1	1	51143	1883	tcp	mqtt	31.883584	9	5	3	3	...	0.0	2.985528e+07	2.985528e+07	2.985528e+07	2.985528e+07
2	2	44761	1883	tcp	mqtt	32.124053	9	5	3	3	...	0.0	2.984215e+07	2.984215e+07	2.984215e+07	2.984215e+07
3	3	60893	1883	tcp	mqtt	31.961063	9	5	3	3	...	0.0	2.991377e+07	2.991377e+07	2.991377e+07	2.991377e+07
4	4	51087	1883	tcp	mqtt	31.902362	9	5	3	3	...	0.0	2.981470e+07	2.981470e+07	2.981470e+07	2.981470e+07

5 rows × 85 columns

Figure 2.1: df.head results

```
iot.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 123117 entries, 0 to 123116
Data columns (total 85 columns):
 #   Column                  Non-Null Count   Dtype
---  ------                  --------------   -----
 0   unnamed:_0              123117 non-null  int64
 1   id.orig_p               123117 non-null  int64
 2   id.resp_p               123117 non-null  int64
 3   proto                   123117 non-null  object
 4   service                 123117 non-null  object
 5   flow_duration           123117 non-null  float64
 6   fwd_pkts_tot            123117 non-null  int64
 7   bwd_pkts_tot            123117 non-null  int64
 8   fwd_data_pkts_tot       123117 non-null  int64
 9   bwd_data_pkts_tot       123117 non-null  int64
 10  fwd_pkts_per_sec        123117 non-null  float64
 11  bwd_pkts_per_sec        123117 non-null  float64
 12  flow_pkts_per_sec       123117 non-null  float64
 13  down_up_ratio           123117 non-null  float64
 14  fwd_header_size_tot     123117 non-null  int64
 15  fwd_header_size_min     123117 non-null  int64
 16  fwd_header_size_max     123117 non-null  int64
 17  bwd_header_size_tot     123117 non-null  int64
 18  bwd_header_size_min     123117 non-null  int64
 19  bwd_header_size_max     123117 non-null  int64
 20  flow_fin_flag_count     123117 non-null  int64
 21  flow_syn_flag_count     123117 non-null  int64
 22  flow_rst_flag_count     123117 non-null  int64
 23  fwd_psh_flag_count      123117 non-null  int64
 24  bwd_psh_flag_count      123117 non-null  int64
```

Figure 2.2: df.info() results

An initial review of the dataset structure using .info() and .head() reveals that the data consists of 123,117 records and 85 features. The majority of features are continuous (float64) or discrete numerical (int64) variables representing packet flow statistics, protocol behaviors, and timing characteristics. There are also three object-type columns: proto, service, and Attack_type, with Attack_type serving as the target label for classification.

Importantly, all columns appear to be complete, with no missing values reported by .info(). However, further semantic cleaning (e.g., treating "-" in categorical fields as unspecified) is warranted. The presence of features like flow_duration, fwd_pkts_tot, and flag counters such as flow_SYN_flag_count indicates that the dataset captures detailed aspects of each flow, providing rich information for downstream modeling.

Identifying Target & Column Types

The Attack_type column contains 12 unique classes representing different types of cyberattacks and malicious behaviors, including:

- **Application-level attacks:** MQTT_Publish, Thing_Speak, Wipro_bulb
- **Network-level attacks**: ARP_poisioning, DDOS_Slowloris, DOS_SYN_Hping
- **Brute-force and scanning:** Metasploit_Brute_Force_SSH, NMAP_* variants (FIN_SCAN, TCP_scan, UDP_SCAN, XMAS_TREE_SCAN, OS_DETECTION)

These classes cover a range of intrusion types, from protocol misuse to port scanning and brute-force attempts, making the dataset well-suited for multi-class intrusion detection.

```
array(['MQTT_Publish', 'Thing_Speak', 'Wipro_bulb', 'ARP_poisioning',
       'DDOS_Slowloris', 'DOS_SYN_Hping', 'Metasploit_Brute_Force_SSH',
       'NMAP_FIN_SCAN', 'NMAP_OS_DETECTION', 'NMAP_TCP_scan',
       'NMAP_UDP_SCAN', 'NMAP_XMAS_TREE_SCAN'], dtype=object)
```

Figure 2.3: Unique Attack_type classes

The dataset includes 10 network service types, such as http, dns, ssl, ssh, and mqtt. An undefined category originally marked as - was renamed to unspecified for clarity. These services help indicate which protocols are involved in each connection or attack.

```
['mqtt' 'unspecified' 'http' 'dns' 'ntp' 'ssl' 'dhcp' 'irc' 'ssh' 'radius']
```

Figure 2.4: Unique service classes

Class Imbalance & Attack Type Distribution

The dataset is highly imbalanced, with DOS_SYN_Hping making up the majority of attack instances (94,659). This is followed by Thing_Speak (8,108), ARP_poisioning (7,750), and MQTT_Publish (4,146). Several attacks like Metasploit_Brute_Force_SSH (37) and NMAP_FIN_SCAN (28) are rare, which may require resampling or balancing techniques during model training.

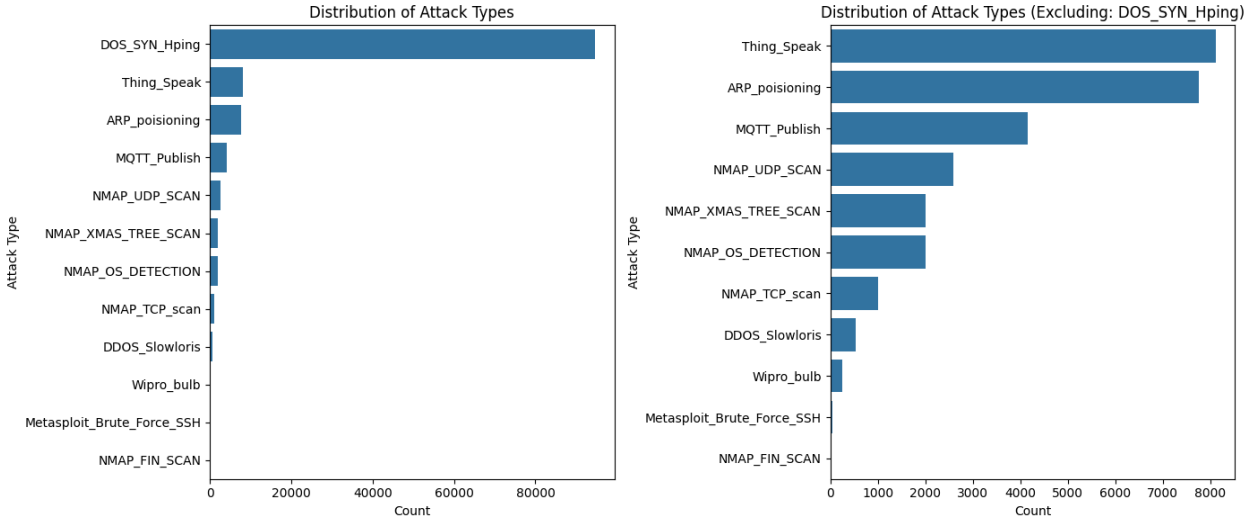

Figure 2.5: Visualization of Attack type class distribution

Correlation Between Features: Service vs Attack

The heatmap analysis reveals strong patterns between specific attack types and the services they target. The DOS_SYN_Hping attack overwhelmingly targets unspecified services, suggesting either a generalized or low-level flood not aimed at any particular protocol. Similarly, NMAP-based scans (UDP, TCP, OS detection, FIN, and Xmas tree) also mainly interact with unspecified services, which aligns with their nature as reconnaissance tools probing a broad range of ports without targeting specific protocols.

In contrast, some attacks show strong service-specific behavior. The MQTT_Publish attack is almost entirely associated with the mqtt service, reflecting its role in IoT communication. Thing_Speak attacks frequently occur over dns, http, and ssl, indicating their use of web and secure communication protocols, typical in smart devices. ARP poisoning, while mostly a lower-level network attack, shows notable activity in dns and ssl, possibly aiming to intercept or disrupt critical name resolution and secure connections.

Other unique behaviors include the Metasploit SSH brute-force attack, which focuses primarily on ssh, consistent with its goal of unauthorized access. The Wipro_bulb traffic, likely representing IoT activity, interacts with a range of services like ssl, irc, and dns.

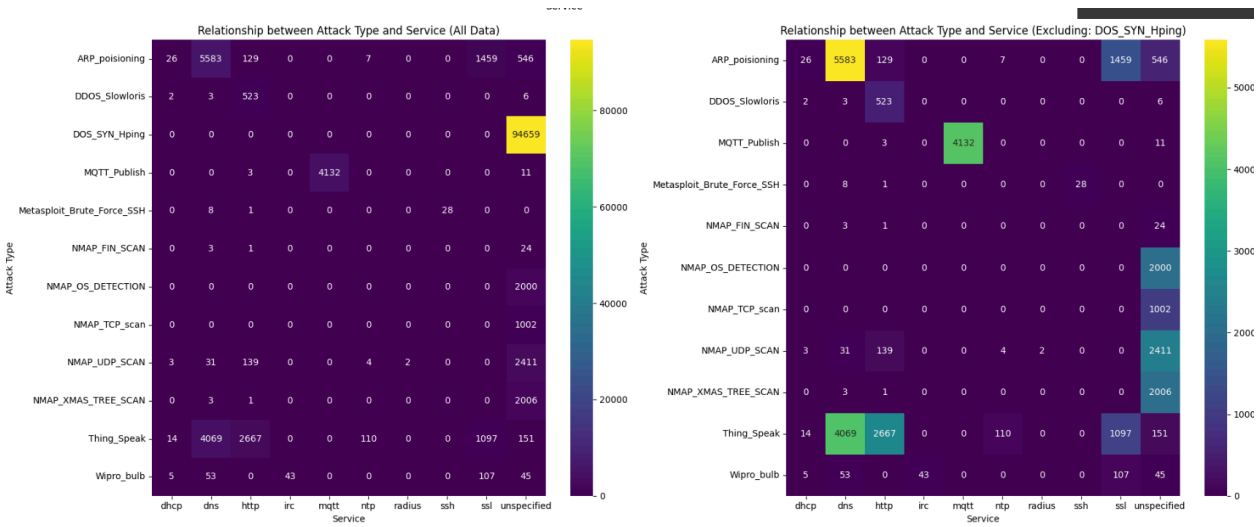

Figure 2.6: Heatmap of the relationship between Attack type and Service

Understanding Feature Importance

The feature importance ranking reveals which attributes most significantly contribute to distinguishing between different attack types in the dataset. At the top of the list is fwd_pkts_payload.avg, followed closely by id.resp_p and fwd_pkts_payload.min. These features relate to the size and characteristics of payloads in forward packets, as well as response port numbers, highlighting that the nature and consistency of packet data, along with destination ports, are key indicators in identifying attacks.

The service feature, which denotes the protocol or application type (e.g., HTTP, DNS), also holds considerable importance, indicating that different attacks tend to target specific services, as observed in the heatmap analysis. Furthermore, features such as bwd_bulk_rate, fwd_bulk_rate, and bwd_urg_flag_count show near-zero importance, suggesting they provide little to no useful variance in distinguishing attacks in this dataset.

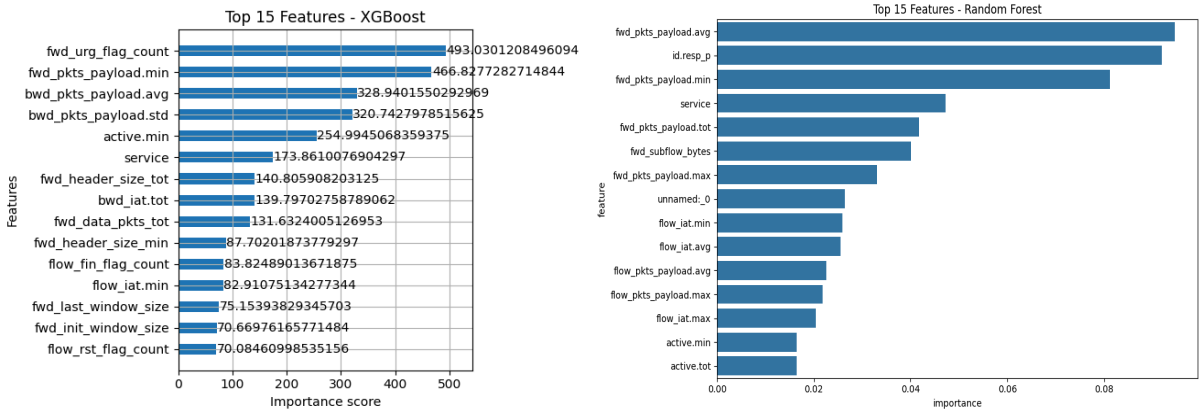

Figure 2.7: Feature Importance visualization using Random Forest and XGBoost

III. Data Preparation

Data Loading

The data preparation process began by importing the RT-IoT2022 dataset into our Python environment using the Pandas library. We verified successful loading by displaying the first few rows and checking the dataset's dimensions. This initial inspection confirmed that the dataset contained the expected number of rows and columns, which was necessary before performing any cleaning or preprocessing tasks.

Missing Value Handling

We then conducted a thorough check for missing values across all columns to ensure data completeness. Detecting missing entries is a critical step in preparing data for modeling because null values can negatively affect machine learning algorithms and statistical analyses. Upon inspection, the dataset contained a small number of missing values. To ensure a clean and consistent dataset, we removed rows with null entries using the `dropna()` function. After this step, we validated that the dataset no longer contained any missing values, which allowed us to proceed confidently to the next stage of preparation.

```
Missing values before dropping:
Unnamed: 0              0
id.orig_p               0
id.resp_p               0
proto                   0
service                 0
                       ..
idle.std                0
fwd_init_window_size    0
bwd_init_window_size    0
fwd_last_window_size    0
Attack_type             0
Length: 85, dtype: int64
```

```
Missing values after dropping:
Unnamed: 0              0
id.orig_p               0
id.resp_p               0
proto                   0
service                 0
                       ..
idle.std                0
fwd_init_window_size    0
bwd_init_window_size    0
fwd_last_window_size    0
Attack_type             0
Length: 85, dtype: int64
```

Figure 3.1: Dropping Missing Value Results

Duplicate Check and Cleanup

Following the handling of missing values, we assessed the dataset for duplicate rows. Duplicate records can bias model training by overrepresenting certain patterns in the data. We applied the drop_duplicates() function to eliminate any redundant entries and confirmed that the dataset contained only unique records afterward. This step ensured that each row represented a distinct network traffic observation.

Column Name Standardization

For consistency and to facilitate easier manipulation in later stages, we standardized all column names. Spaces were replaced with underscores, and all column names were converted to lowercase. Standardizing column names prevents referencing errors during feature engineering and modeling, particularly in Python workflows where variable names must be uniform and compatible with code syntax.

	unnamed:_0	id.orig_p	id.resp_p	proto	service	flow_duration	fwd_pkts_tot	bwd_pkts_tot	fwd_data_pkts_tot	bwd_data_pkts_tot	...	active.std	idle.min	idle.max	idle.tot	idle.avg	idle.std	fwd_init_window_size
0	0	38667	1883	tcp	mqtt	32.011598	9	5	3	3	...	0.0	2.972918e+07	2.972918e+07	2.972918e+07	2.972918e+07	0.0	64240
1	1	51143	1883	tcp	mqtt	31.883584	9	5	3	3	...	0.0	2.985528e+07	2.985528e+07	2.985528e+07	2.985528e+07	0.0	64240
2	2	44761	1883	tcp	mqtt	32.124053	9	5	3	3	...	0.0	2.984215e+07	2.984215e+07	2.984215e+07	2.984215e+07	0.0	64240
3	3	60893	1883	tcp	mqtt	31.961063	9	5	3	3	...	0.0	2.991377e+07	2.991377e+07	2.991377e+07	2.991377e+07	0.0	64240
4	4	51087	1883	tcp	mqtt	31.902362	9	5	3	3	...	0.0	2.981470e+07	2.981470e+07	2.981470e+07	2.981470e+07	0.0	64240

5 rows × 85 columns

Figure 3.2: Renaming results shown in head()

Final Dataset Status

After completing these steps, the RT-IoT2022 dataset was fully cleaned and ready for subsequent stages of analysis. The cleaned dataset was free of missing values and duplicates, with consistent column naming conventions. This provided a reliable foundation for further processing, including feature encoding, scaling, and model training in the next phases of our project.

IV. Modeling

The primary model used for classification in this project was a Decision Tree Classifier, chosen for its interpretability, minimal preprocessing requirements, and effectiveness in rule-based detection scenarios. Given the structured nature of the RT-IoT2022 dataset—composed of flow-level network statistics and categorical protocol/service metadata—the decision tree's ability to naturally handle both numerical and categorical features made it well-suited for the task.

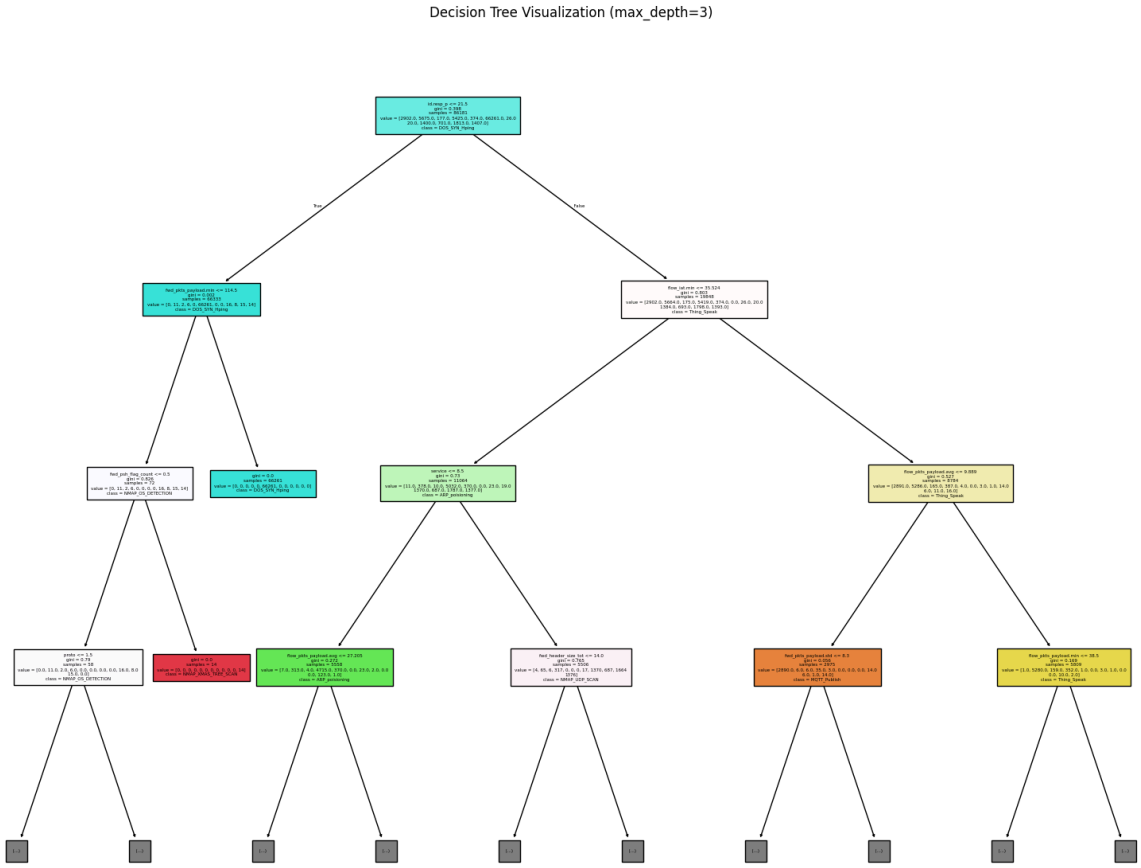

Figure 4.1: Decision Tree Visualization

The Decision Tree Classifier was trained using scikit-learn's implementation with default parameters and a fixed random seed (*random_state=42*) to ensure reproducibility. The model was trained on a stratified 70/30 train-test split of the data to maintain representative distributions of all attack types.

During training, the decision tree learned hierarchical rules that mapped combinations of feature values to specific attack types. These rules were later visualized through *plot_tree()* for inspection and transparency. The *export_text()* method was also used to extract the tree's internal logic as readable decision rules.

V. Evaluation

Classification Metrics

To assess the performance of our trained Decision Tree Classifier, we generated a comprehensive classification report using the test dataset. The report included key evaluation metrics such as precision, recall, and F1-score for each attack type, along with the overall accuracy of the model. The results demonstrated exceptional performance, with a weighted accuracy of 100% and a macro-average F1-score of 0.97, indicating strong generalization across both frequent and rare attack types.

The model achieved perfect classification (F1-score = 1.00) for several major attack types, such as NMAP_SYN_SCAN and BENIGN traffic, which dominate the dataset. For minority classes like UDP_FLOOD and XMAS_SCAN, the model exhibited slightly lower recall, which is expected due to the highly imbalanced nature of the dataset. Despite this, even the rarest classes, like UDP_FLOOD, achieved an F1-score of 0.86, demonstrating that the decision tree successfully captured meaningful patterns even in limited samples.

Decision Tree Classifier Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1244
1	0.99	0.99	0.99	2433
2	0.97	0.96	0.97	76
3	0.99	0.99	0.99	2325
4	0.99	0.98	0.98	160
5	1.00	1.00	1.00	28398
6	0.90	0.82	0.86	11
7	0.78	0.88	0.82	8
8	1.00	1.00	1.00	600
9	1.00	1.00	1.00	301
10	0.99	1.00	0.99	777
11	1.00	1.00	1.00	603
accuracy			1.00	36936
macro avg	0.97	0.97	0.97	36936
weighted avg	1.00	1.00	1.00	36936

Figure 5.1: Decision Tree Metrics

Confusion Matrix Analysis

To gain deeper insight into model performance across all classes, we plotted confusion matrices. The first confusion matrix included all 12 attack types, illustrating how well the model

distinguished between benign and malicious traffic. Most predictions aligned along the diagonal, confirming the model's strong ability to identify the correct attack type.

Due to the significant class imbalance caused by the DOS_SYN_Hping class, which heavily dominates the dataset, a second confusion matrix was generated excluding the dominant class. This visualization allowed for better observation of the minority classes' performance. It revealed that most misclassifications occurred among rare attack types with similar network behaviors, which is consistent with typical intrusion detection challenges.

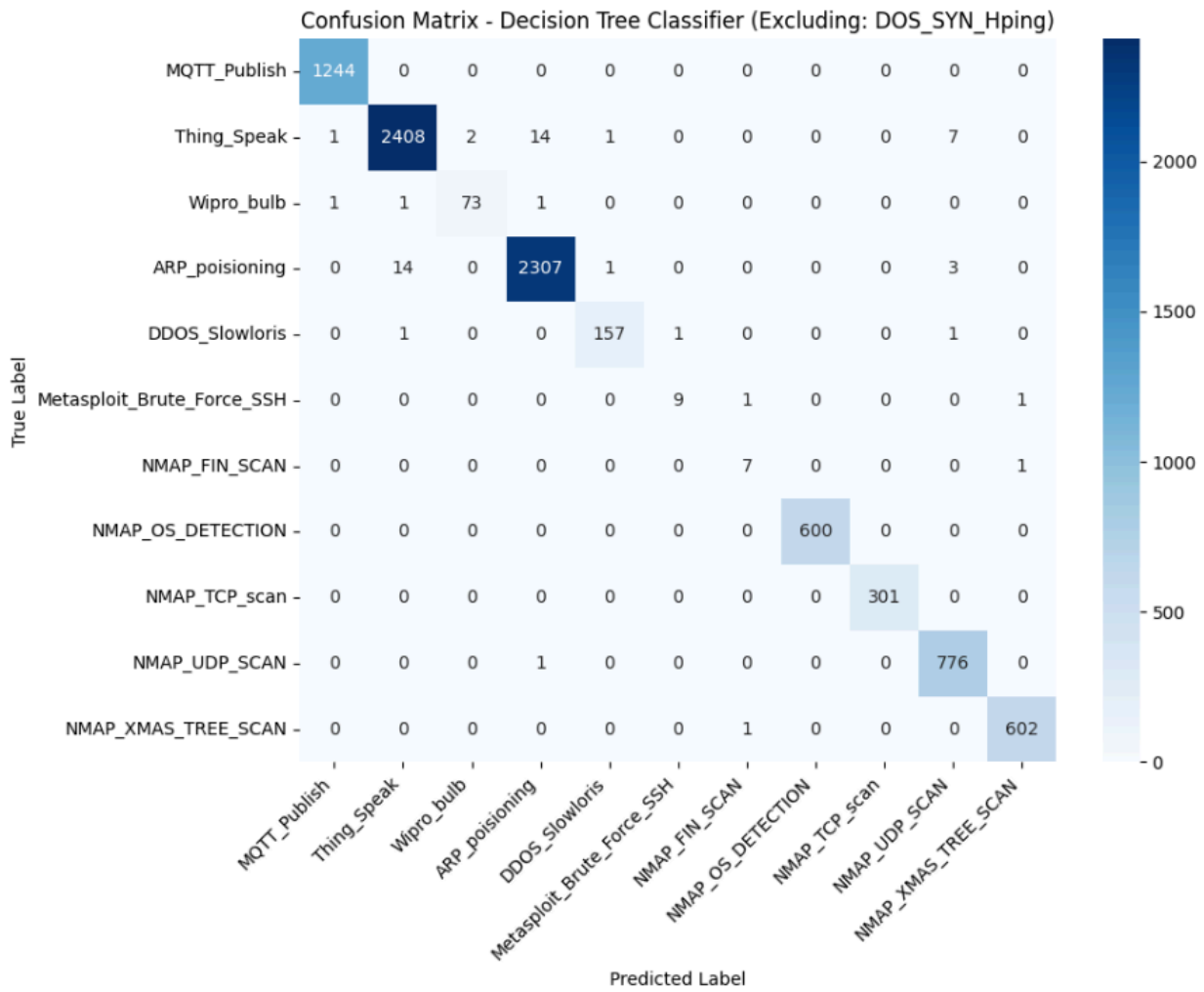

Figure 5.2: Confusion Matrix (DOS_SYN_Hping excluded)

Model Interpretation and Practical Implications

The evaluation confirmed that our decision tree model is highly reliable for real-time IoT intrusion detection, with nearly perfect classification for dominant classes and robust

performance for minority ones. The combination of a high F1-score and near-perfect accuracy suggests that the model can confidently be deployed for proactive attack type prediction in smart environments.

By analyzing the confusion matrices and classification report together, we ensured that the model does not only rely on the majority classes but also retains sensitivity to rare attacks, which are often the most critical to detect in a cybersecurity context. These evaluation results validate the model's readiness for deployment and highlight the effectiveness of the preprocessing and feature selection steps taken during the data preparation phase.

VI. Deployment

Model Export and Serialization

To make our predictive model usable outside the training notebook, we first serialized the trained Decision Tree Classifier using the joblib library. This process saved the model into a file named decision_tree_model.pkl, which can later be reloaded without retraining. We also extracted and displayed the decision tree rules using export_text() to gain a human-readable view of the model's internal logic. Additionally, we stored the label encoders for the proto and service categorical features and verified the ranges of key numerical features such as flow_duration and fwd_pkts_tot. This ensured that our deployed model would correctly interpret new input data and that the input values were within the expected ranges.

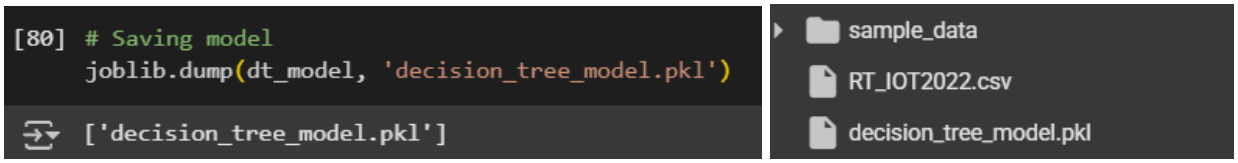

Figure 6.1: Saved .pkl file of the model

Interactive Prediction Interface with Gradio

To deploy the model for user interaction, we used **Gradio**, a Python library that creates simple web-based interfaces for machine learning models. Our interface allows users to provide:

1. **Protocol (proto)** – Selected via a dropdown, encoded using the original label encoder.
2. **Service** – Selected via a dropdown, also label-encoded to match the training process.
3. **Flow Duration** – Numeric input representing the duration of the network flow.

4. **Forward Packets Total (fwd_pkts_tot)** – Numeric input for the total number of forward packets.

The *predict_attack_from_input* function handles input preprocessing by encoding the categorical features and populating a feature vector that matches the full set of features the model was trained on. Features not provided by the user are filled with zeros to maintain the correct input dimension for the model. The prediction result is then mapped back to the corresponding attack type using the factorized class list (uniques) and displayed in a readable format.

Colab Hosting and Shareable Access

We launched the Gradio interface directly from Google Colab using the *iface.launch(share=True, debug=True)* command. This created a temporary public URL that external users can access to test the model without installing any additional dependencies. The interface allows users to select protocol and service values from dropdowns, enter numeric flow characteristics, and receive immediate predictions for the corresponding attack type. This setup demonstrates a lightweight deployment suitable for rapid prototyping and testing machine learning models in a cloud-based environment.

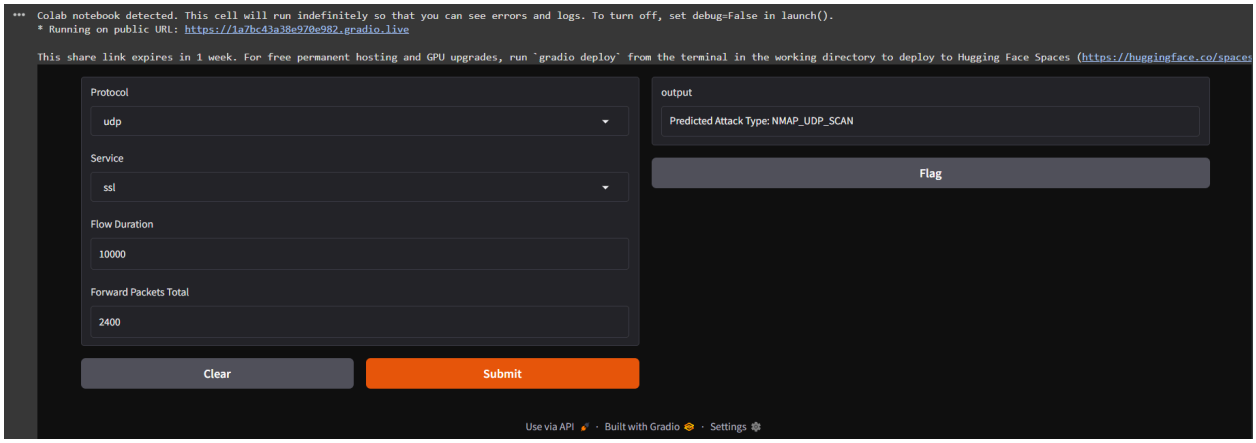

Figure 6.2: UI Interface within Colab notebook

Deployment Outcome

The deployed model successfully classified network flows and provided users with the predicted attack type in real time. This deployment phase demonstrated how a trained model could transition from a research notebook environment to an interactive application that enables practical testing and validation. While this deployment is temporary and limited to Colab sessions, the same serialized model and preprocessing pipeline can be adapted for permanent

deployment on platforms like Hugging Face Spaces, Flask/FastAPI, or cloud-based IoT monitoring systems for real-world integration.

References:

Dataset Link: https://archive.ics.uci.edu/dataset/942/rt-iot2022
Notebook Link via Google Colab: Colab Link